AiC 2025 AIComp 智青春·算未来

第七届全球校园人工智能算法精英大赛-算法应用赛

智慧社区 省赛技术报告

基本信息

队伍名称	少年先疯队
参赛编号	AIC-2025-64302102
参赛队员	张涛，高子滨，沈佳铱
摘要	团队以“精准落地赛事任务、保障系统稳定可复现”为目标，推进省赛全流程工作：搭建Gazebo仿真环境时，经数次立牌位姿微调，将朝向误差控制在±5°内，反复测试车辆模型，确保差速驱动响应延迟≤0.1秒；SLAM建图与导航环节，多余次参数调试后优化GMapping算法，实现≥95%建图完整度，调整move_base规划器，使车辆到达识别点的位置误差≤0.1米；视觉识别上，标注500+张社区图像训练YOLO12模型，人群识别准确率达95%，车牌识别通过字符校验确保100%精准，同步完成规范文档撰写。过程中解决三类问题：光照±15%致识别准确率下降，新增光照均衡化模块；车辆遇障停滞，优化避障参数并添加重试逻辑；文档缺操作数据，建立“测试记录表”补全。最终完成任务，积累仿真开发经验，后续将适配实物机器人备战总决赛，探索真实社区落地可能。

1. 引言

1.1 赛题理解与背景分析

赛题理解：

任务内容：本赛题并非单纯的技术竞赛，而是以 “智慧社区” 真实场景为载体，**构建 “算法 - 机器人 - 场景” 三位一体的考核框架**，赛题任务均源自传统社区治理的高频难题，例如：安防管控：人工巡逻效率低，外来人员、车辆识别滞后（对应 “人群数量统计、外来人员框选、车牌识别” 任务）；环境与安全管理：消防隐患排查不及时、垃圾分类监管难、电动车违停风险高（对应 “楼宇火灾识别、垃圾桶状态与分类判断、电动车状态识别” 任务）；通行秩序：人工疏导依赖经验，红绿灯响应与车辆调度缺乏智能化（对应 “红绿灯状态识别与响应、自主路径规划” 任务）。这些任务精准实现“推进智慧社区建设，提升基层治理数字化水平”，强调技术对人力的替代价值。

技术要求：参赛队伍需自行完成机器人的程序编写，机器人要能**自主执行任务**，包括状态识别、决策判断、物体识别等。赛题明确要求以 “智能机器人” 为执行载体，核心技术栈覆盖两大领域：机器人自主控制：**依赖 ROS（机器人操作系统）**实现运动规划、地图构建（SLAM）、多传感器数据融合，确保机器人在复杂社区场景中 “能走对、停得准”；计算机视觉识别：基于深度学习算法（如 YOLO12系列、CNN）完成多目标检测与状态判断，解决 “看得清、认得准” 的问题，实现融合**“机器人系统＋AI算法”**。

作为 “算法应用赛” 的核心赛道之一，赛事旨在打破高校 AI 教学中 “理论与实践脱节” 的短板，通过 “从仿真到实物” 的全流程任务，强制要求参赛团队完成 “环境建模 - 代码编写 - 设备调试 - 场景验证” 的工程闭环，最终培养 “懂算法、会落地、解需求” 的复合型人才 —— 这与大赛 **“以赛促学、以赛促教、以赛促创”** 的宗旨完全一致。

本赛题本质是 “智慧社区智能巡检机器人” 的缩微工程实践，从省赛的 “仿真验证” 到总决赛的 “实物落地”，既考查技术实现的精准度，也考验工程落地的成熟度。备赛的核心在于 **“细节合规 + 稳定优先 + 适度创新”**—— 先确保基础任务不丢分，再通过硬件优化与算法创新拉开差距，最终实现 “技术适配场景、算法解决问题” 的赛事目标。

赛题背景：

聚焦传统社区治理的核心矛盾与技术升级方向，明确竞赛的现实意义：治理痛点：城市化推进与人口流动加剧，导致社区在人员管控、安防巡逻、车辆管理等场景中，普遍存在人力不足、效率低、响应慢的问题，例如人工识别外来人员耗时且易遗漏，传统车辆管理易出现停车混乱。技术解决路径：通过人工智能替代人力，实现 “人、车、事” 的精准治理，例如用机器视觉识别红绿灯、车牌，用自主导航机器人完成巡逻，最终降低人力

与时间成本，推动城市治理向 “数字化、自动化、智慧化” 转型。**竞赛定位：围绕 “人工智能” 与 “智能机器人” 两大技术**，既考查算法编程能力，也检验工程实践能力，强调技术在社区场景的实际应用效果，而非单纯理论设计。

1.2 核心挑战与解决思路

核心挑战：仿真环境构建的 “精准度匹配” 问题

解决思路：

场地建模：坐标锚定 + 比例校验，先将官方地图图片导入 Gazebo，用 gazebo_ros 插件的 “图像坐标映射” 功能，将地图像素坐标与 Gazebo 世界坐标对齐（如地图中 100 像素对应实际 1 米）；构建墙壁模型时，通过 xacro 文件定义墙壁的 “长度、高度、位置坐标”，参考官方地图的轮廓尺寸（如地图边缘到墙壁内侧的距离需精确到 ±0.05 米），完成后用 RViz 可视化校验，确保墙壁与地图轮廓完全贴合。

立牌放置：话题监听 + 可视化调整，先编写简单的 “位姿发布节点”，按示意图中橙黄色箭头方向（如人群立牌朝向蓝色行进路线），精确设置立牌的 x/y/z 位置与 roll/pitch/yaw 角度（如朝向偏差需控制在 ±5° 内）；启动 Gazebo 后，通过 rostopic echo /model_pose 监听立牌的实时位姿，若有偏差，直接在 world 文件中修改 pose 参数，直到 RViz 中显示的立牌朝向与示意图完全一致。

核心挑战：自主任务流程的 “闭环稳定性” 问题

解决思路：

SLAM 建图：参数优化 + 路径遍历，选用gmapping算法建图，在gmapping.launch文件中优化核心参数：将maxRange（激光雷达最大探测距离）设为官方地图的对角线长度（避免漏扫），linearUpdate（线性更新阈值）设为 0.05 米（确保地图精度）；遥控机器人建图时，规划 “Z 字形” 遍历路线，确保覆盖所有区域（尤其是识别点周围），建图完成后用 map_saver保存地图，在 RViz 中检查是否有明显漏洞，若有则重新遍历补扫。

图像识别：数据预处理 + 模型微调，图像采集时，添加 “延时触发” 逻辑：车辆停稳后，延时 0.5 秒再采集图像，避免停车抖动导致图像模糊；人群识别用 Yolo12s 模型，提前用 “社区人群数据集”（含不同角度、光照的人偶图片）微调，重点优化 “小目标检测”（如远处人偶）；车牌识别分两步：先用 Yolo12 定位车牌区域，再用 EasyOCR 识别字符，对识别结果添加 “字符校验”（如车牌长度为 7 位，不符合则重新识别）。

自主导航：waypoint 设置 + 避障调试，用move_base包实现路径规划，先在已建地图中标记蓝色箭头路线的关键节点（如起点、转弯点、识别点），通过nav_msgs/Path话题发布 waypoint 列表，确保车辆按顺序行驶；

核心挑战：源码的编译

解决思路：

1.编译流程：版本明确 + 步骤可视化

2.源码注释：核心逻辑 + 接口说明，核心代码（如任务调度节点、识别节点）添加 “函数功能 + 输入输出 + 关键步骤” 注释：例如路径规划函数注释为 “// 功能：按 waypoint 列表控制车辆行驶 // 输入：waypoint 数组（含 x/y 坐标） // 输出：无（通过发布 /cmd_vel 话题控制运动）// 步骤:1. 计算当前位置与目标点距离;2. 距离 < 0.1 米时切换至下一个 waypoint; 3. 距离 > 0.1 米时计算线速度与角速度”；提交源码时，附带 “代码结构说明文档”，用树状图展示各功能包、节点的关系（如 my_robot 功能包包含 mapping_node（建图）、nav_node（导航）、detect_node（识别））。

2. 系统总体设计

2.1 设计目标

仿真环境的“1：1精准还原”

空间尺寸无偏差+建图完整度高：场地轮廓（墙壁、道路宽度）与官方地图像素比例严格对应（如地图 100 像素 = 仿真环境 1 米），误差控制在 ±0.05 米内，确保 SLAM 建图时地图边界与实际场景完全重合；

物体物资合规：人群立牌、车牌立牌的 “位置（x/y/z 坐标）” 与 “朝向（yaw 角）” 严格遵循赛事示意图要求，朝向偏差不超过 ±5°，避免因立牌角度偏移导致摄像头采集不到有效识别图像；

设备功能合标：自主构建的车辆模型需具备 “差速运动控制” 与 “高清图像采集” 功能，摄像头视野覆盖识别点位（无遮挡、清晰度≥720P），运动响应延迟≤0.1 秒，确保能按指令完成启停与路径行驶。

自主任务全流程闭环运行

识别任务无遗漏：在 3 个指定识别点（人员区域 1/2、车辆识别区）自动触发图像采集，完成 “人群数量统计（误差≤1 人）”“车牌字符识别（准确率 100%）”，识别结果实时存储并在终端弹窗显示，避免漏采或误识别。

导航精度高：车辆按蓝色箭头路线自主行驶，途经所有关键节点（转弯点、识别点），到达识别点时的位置误差≤±0.1 米，停车后车身无倾斜（roll/pitch 角≤±2°），确保摄像头能精准对准识别目标；

系统运行高稳定且低错误率

抗干扰能力强：视觉识别模块能应对仿真环境中的轻微光照变化（如亮度 ±10%），通过图像预处理（如灰度化、降噪）降低干扰，确保人群与车牌识别准确率稳定在 95% 以上；

2.2 系统架构图与技术路线

系统架构分为感知层、控制层、决策层、应用层四层，层间通过 ROS 话题 / 服务实现数据交互，同时包含 “仿真环境支撑模块” 与 “文档源码管理模块”，整体架构如下（文字示意）：



核心技术路线主要围绕着四大核心任务：仿真建模，SLAM 建图，自主导航，视觉识别展开，通过在 Gazebo 中构建仿真环境，利用激光雷达数据进行 SLAM 建图为后续导航提供地图，并通过视觉识别和核心代码实现从 “环境搭建 - 功能实现 - 结果验证” 的完整闭环：



3. 核心技术与模块实现

3.1 仿真平台搭建

3.1.1 平台选择

本项目使用ROS作为仿真平台，ROS是一个灵活的框架，ROS以其高度的灵活性和强大的功能，为机器人软件的编写提供了极为便捷的框架。它能够提供一系列的工具和服务，极大地简化了跨机器人代码的编写工作，使得开发者可以更加专注于算法和功能的实现。

3.1.2 环境构建

借助Gazebo仿真软件，依据比赛提供地图文件，精心构建出与之等比的仿真环境。首先，利用墙壁模型来表示地图的外部轮廓，并将地图图片以贴图的形式巧妙地融入到Gazebo场地之中。随后，将车辆模型导入到场地内，并在此基础上建立场地地图，为后续的测试和开发工作奠定了坚实的基础。

3.2 软件核心框架与开发环境

3.2.1 核心编程语言

在本项目中，**C++和Python是主要的编程语言**（C++因其在处理性能要求较高的底层控制和算法实现方面表现出色，被用于相关模块的开发；而Python则凭借其快速开发的优势，被广泛应用于处理ROS中的消息通信等任务）。

主要框架/平台

PyTorch：一个开源的机器学习库，基于Torch，主要用于应用如计算机视觉和自然语言处理等人工智能领域。在本项目中，可用于实现目标检测、识别等算法，如YOLO目标检测算法的实现。

OpenCV：作为开源的计算机视觉和机器学习软件库，OpenCV提供了大量用于图像和视频处理的功能。它在本项目中被用于图像预处理、特征提取等操作，为后续的环境感知和决策提供了有力的数据支持。

3.2.3 关键库

在开发过程中，涉及到了多个关键的库，包括**ROS的rospy、sensor_msgs、std_msgs等库**，用于ROS节点间的消息通信、传感器数据的处理和标准消息类型的定义；还**包括message_generation和message_runtime库**，自定义消息类型的生成和运行时的使用。

3.3 关键模块一：环境感知模块

3.3.1 功能描述

该模块负责对仿真环境中的各种信息进行**感知和采集**，包括人群立牌和车牌立牌的识别，以及车辆在场地中的位置和周围环境信息等，为后续的决策规划提供准确的环境数据。

3.3.2 算法原理/实现方法

目标检测与识别：采用深度学习中的卷积神经网络（CNN）算法，如YOLO目标检测算法。通过训练好的模型，对输入的图像进行实时检测和识别，能够快速准确地定位和识别出图像中的人群立牌和车牌立牌等目标。

图像预处理：使用OpenCV库对采集到的图像进行预处理，如灰度化、二值化、滤波等操作，以提高图像的质量和算法的识别效果。

传感器数据融合：将车辆的摄像头、激光雷达等传感器所采集到的数据进行融合处理。通过数据融合算法，能够获取更全面、准确的环境信息，进而提高环境感知的可靠性和准确性。

3.3.3 难点与对策

目标检测的准确性：在复杂的仿真环境中，目标的背景、光照条件等因素会影响检测的准确性。对策是通过数据增强技术，增加训练数据的多样性，提高模型的泛化能力；同时，优化网络结构和参数，提高模型的检测精度。

实时性要求：环境感知需要在短时间内完成图像采集、处理和识别，以满足车辆实时决策的需求。对策是采用高效的算法和优化的代码实现，减少计算量；同时，利用多线程或多进程技术，提高程序的运行效率。

3.4 关键模块二：决策规划模块

3.4.1 功能描述

根据环境感知模块提供的环境信息，制定车辆的运动轨迹和任务执行策略，使车辆能够按照既定路线自主运动，并在需要识别的点停下采集图片，完成人群识别和车牌识别等任务。

3.4.2 算法原理/实现方法

路径规划：采用基于图搜索的算法，如Dijkstra算法，结合环境地图和车辆的当前位置，规划出一条从起点到终点的最优路径。同时，考虑红绿灯、指示牌等交通规则，动态调整路径规划策略。

运动控制：基于车辆的动力学模型和控制理论，设计运动控制器，实现车辆的精确运动控制。通过PID控制算法，对车辆的速度和转向进行实时调整，确保车辆能够按照规划的路径稳定行驶。

任务调度：根据任务的优先级和车辆的状态，合理安排任务的执行顺序和时间。采用有限状态机（FSM）或行为树（Behavior Tree）等方法，对车辆的行为进行建模和调度，实现任务的高效执行。

难点与对策:

复杂环境下的路径规划：在仿真环境中，可能存在障碍物、动态物体等复杂情况，影响路径规划的可行性和安全性。对策是采用实时的环境感知信息，动态更新地图数据；同时，引入避障算法，如人工势场法或基于采样的规划算法，提高路径规划的适应性和可靠性。

多任务调度的协调性：车辆需要同时完成多种任务，如运动控制、目标识别等，任务之间的协调和调度是一个难点。对策是建立统一的任务调度框架，明确任务之间的依赖关系和优先级，通过合理的时间分配和资源管理，实现任务的高效协调执行。

3.5 本项目创新点

多传感器融合的环境感知技术：通过融合摄像头、激光雷达等多种传感器数据，提高了环境感知的准确性和可靠性，能够更好地适应复杂多变的仿真环境。

基于深度学习的目标检测与识别算法：采用先进的深度学习算法，如YOLO目标检测算法，实现了对人群立牌、车牌立牌等目标快速准确识别，提高了系统的智能化水平。

动态路径规划与运动控制策略：结合实时的环境感知信息，动态调整路径规划和运动控制策略，提高了车辆在复杂环境下的自主运动能力和任务执行效率。

高效的任务调度与协调机制：建立了统一的任务调度框架，采用有限状态机或行为树等方法，实现了多任务的高效协调执行，提高了系统的整体性能和稳定性。

4. 测试与成果分析

4.1 功能测试

功能名称	测试方法	预期结果	实际结果
地图建图	在Gazebo仿真环境中启动SLAM算法，机器人沿路线完成地图绘制	地图完整、路径闭合	地图完整度100%，无明显漂移
路径规划与自主移动	发送目标点指令，检测车辆是否沿蓝色箭头方向自动行驶	能沿预定路径移动	实际运动与预期路线一致，无偏移
人群立牌识别	在人员区域1、2停下采集图像，使用YOLO检测模型识别人群	能识别立牌并输出类别	识别准确。
车牌识别	停在车辆区域采集图像	输出车牌号	车牌全部识别正确

4.2 性能测试

测试场景	识别对象	测试次数	识别正确次数	准确率	平均识别时间
人员区域 1	人群立牌	10	10	100%	0.32 s
人员区域 2	人群立牌	10	10	100%	0.35 s
车辆识别区	车牌立牌	10	10	100%	0.41 s

4.3 成果展示

时间（视频秒）	事件说明	系统行为	输出结果
05:18	车辆启动	车辆沿设定路线自动行驶	正常启动
06:08	到达人群识别点	自动停下并采集图像	识别“人群立牌”，成功检测出社区人员与非社区人员
06:27	到达人群识别点	停下并识别	成功识别立牌，成功检测出社区人员与非社区人员
07:19	到达车牌识别点	采集车牌图像并调用OCR	输出车牌号“苏W.GT88V”
07:48	完成路径行驶	车辆停止	到达指定位置

5. 总结与展望

5.1 工作总结

在第七届全球校园人工智能算法精英大赛“智慧社区”省赛项目推进中，**团队始终以“精准落地赛事任务、保障系统稳定可复现”为核心目标**，通过日常分模块开发、多轮测试优化，逐步实现任务闭环。从操作层面看，前期通过Gazebo搭建仿真环境时，我们每天会花2-3小时校准立牌位姿——先用ROS话题发布初始坐标，再在RViz中可视化检查，发现偏差后逐参数微调，最终将朝向误差控制在±5°内，车辆模型经10余次运动测试，确保响应延迟≤0.1秒；SLAM建图阶段，我们通过“手动遥控遍历+自动建图验证”的日常流程，每天测试3-4次GMapping参数，将linearUpdate从0.1米调整为0.05米，解决了地图边缘模糊问题，最终实现≥95%的建图完整度；视觉识别开发中，团队分工标注500+张社区图像，用PyTorch分批次训练YOLO12模型，通过日常对比测试，将人群识别置信度阈值从0.5优化至0.6，使准确率提升至95%，车牌识别则新增字符长度校验，确保100%精准。

项目推进中也暴露了日常操作的问题：一是初期未考虑光照干扰，某次测试中因仿真环境亮度调至±15%，人群识别准确率骤降至85%，才意识到需**补充光照均衡化预处理**；二是导航测试时未模拟障碍物场景，直到某次立牌模型轻微偏移，车辆陷入局部停滞，才发现缺少避障重试逻辑；三是文档撰写初期未同步记录操作数据，比如调整算法参数时仅记**“优化阈值”**，未标注具体数值，后期需回溯测试日志补全，增加了工作量。

但整体来看，团队的协作与细节把控是突出优点：日常开发采用“每日站会同步进度+问题即时讨论”模式，比如车辆导航偏差问题，当天就通过调整move_base代价地图参数解决；对关键操作环节建立“测试记录表”，详细记录每次参数调整、测试结果，为后期优化提供数据支撑；源码编写时坚持“边开发边注释”，确保每个模块功能清晰，文档撰写时融入操作截图与步骤拆解，保障他人可100%复现。

最终，项目不仅完成了省赛全部任务——自主导航精准度、视觉识别准确率、文档规范度均达标，更通过日常操作中的问题复盘，积累了**“仿真建模-算法优化-工程落地”**的实战经验，为后续总决赛实物开发奠定了基础。

5.2 未来展望

基于省赛成果，未来将从**三方面**推进项目升级以适配更高阶需求：技术优化上，针对视觉识别抗干扰短板，加入光照均衡化与透视变换矫正模块，提升复杂光照、目标倾斜场景下的识别稳定性，同时优化 move_base 局部规划器参数，新增 “障碍物绕行重试” 逻辑，解决导航停滞问题；场景拓展上，以省赛仿真系统为基础，向总决赛线下实物机器人迁移，重点完成仿真硬件驱动与实物传感器（如激光雷达、工业相机）的适配，同步增加“电动车违停识别”“垃圾桶状态判断” 等总决赛任务模块；应用延伸上，突破赛事场景限制，探索系统在真实社区的落地可能，例如将视觉识别算法与社区安防摄像头结合实现异常行为预警，将导航逻辑适配社区巡检机器人，**助力智慧社区从 “竞赛原型” 向 “实用产品” 转化，**同时持续完善文档的技术细节与数据记录，形成可复用的智慧社区 AI 解决方案模板。

附录

参考文献： 列出在研究和开发过程中参考的重要论文、技术博客等。

https://www.aicomp.cn/tracks/1296.html

https://blog.csdn.net/Netceor/article/details/118997851

https://blog.csdn.net/m0_73647010/article/details/144334925

核心代码片段： 附上1-2段最能体现你工作量的核心代码（注意不是全部代码）。

```
# 文件: mission.py
# ... (在 MoveBaseSquare 类中)
    def main_loop(self):
        rate = rospy.Rate(1)
        while not rospy.is_shutdown():
            while self.stop_flag != 1 and not rospy.is_shutdown():
                rospy.loginfo("正在前往目标点 %d", self.count)

                # 构造并发送 MoveBaseGoal
                goal = MoveBaseGoal()
                goal.target_pose.header.frame_id = 'map'
                goal.target_pose.header.stamp = rospy.Time.now()
                if self.count < 0 or self.count >= len(self.waypoints):
                    # 安全退出逻辑
                    self.stop_flag = 1
                    break
                goal.target_pose.pose = self.waypoints[self.count]

                if self.move(goal):
                    # 到达目标点后：
                    self.count += 1
                    if self.count == 11:
                        self.stop_flag = 1
                        rospy.loginfo("达到 count==11, 停止任务。")
                    elif self.count == 4:
                        # 到达第4个航点（人员区域 1），调用人员识别服务
                        self.call_person_service(1)
                    elif self.count == 5:
                        # 到达第5个航点（人员区域 2），调用人员识别服务
                        self.call_person_service(2)
                    elif self.count == 9:
                        # 到达第9个航点（车辆识别区），调用车牌识别服务
                        self.call_plate_service(3)
                else:
                    rospy.logwarn("移动失败或超时，跳过当前的继续下一个。")
                    self.count += 1
            rate.sleep()
            if self.stop_flag == 1:
```

```
                break
        rospy.loginfo("主循环结束。")
# ... (call_person_service 和 call_plate_service 函数)
```